

WHAT IS FEATURE ENGINEERING?

ফিচার ইঞ্জিনিয়ারিং কী?

Feature engineering হলো raw data থেকে meaningful features তৈরি করা, transforming করা অথবা selecting করার পুরো প্রক্রিয়া। এর মূল উদ্দেশ্য হলো machine learning model-এর performance অনেক বাড়িয়ে দেওয়া।

১. Raw Data বনাম Predictive Power

- **Preprocessing:** এর মূল লক্ষ্য হলো ডেটাকে "পরিষ্কার" করা। যেমন: নয়েজ দূর করা, মিসিং ভ্যালু ভরা, বা ডুপ্লিকেট বাদ দেওয়া। এটি অনেকটা রান্নার আগে সবজি ধোয়া বা পরিষ্কার করার মতো।
- **Feature Engineering:** এর লক্ষ্য হলো ডেটার "প্রেডিক্টিভ পাওয়ার" বাড়ানো। অর্থাৎ কলামগুলোকে এমনভাবে সাজানো যাতে মেশিন লার্নিং অ্যালগরিদম সহজে প্যাটার্ন বুঝতে পারে। এটি অনেকটা সবজিগুলোকে কাটা কাটি করে রান্নার জন্য বিশেষ শেপ দেওয়ার মতো।

৩. কেন এটি আলাদা টপিক?

ডেটা মাইনিংয়ে যখন আমরা **Preprocessing** পড়ি, তখন আমরা ডেটার "স্বাস্থ্য" ঠিক করার দিকে নজর দিই। আর যখন **Feature Engineering** পড়ি, তখন আমরা নজর দিই "মডেলের পারফরম্যান্স" বাড়ানোর দিকে।

- **Feature Selection:** তুমি আগে **Attribute Subset Selection** পড়েছ। সেটিই যখন মেশিন লার্নিং মডেলের জন্য করা হয়, তখন তাকে Feature Selection বলা হয়।
- **Feature Extraction:** যেমন **PCA** বা **Wavelet Transform**। এগুলো ডেটা ছোট করে (Reduction), আবার নতুন ফিচারও তৈরি করে (Extraction)।
- **Feature Construction:** তুমি আগে **Attribute Creation** পড়েছ (যেমন: BMI বের করা)। এটিই মূলত Feature Construction।

৪. সহজ উদাহরণ দিয়ে পার্থক্য বোঝা:

ধরো একটি কলামে আছে **Date: 2024-05-20**।

1. **Preprocessing:** যদি এই কলামে কোনো তারিখ ভুল থাকে (যেমন: ৩০শে ফেব্রুয়ারি), তবে সেটা ঠিক করা বা বাদ দেওয়া হলো প্রি-প্রসেসিং।
2. **Feature Engineering:** এই তারিখ থেকে তুমি বের করলে এটা কি **Weekend** নাকি **Weekday**? অথবা এটা কি মাসের **First Week**? মেশিন লার্নিং মডেলের জন্য "তারিখ"-এর চেয়ে "উইকেন্ড" কি না তা জানা বেশি গুরুত্বপূর্ণ হতে পারে। এই নতুন বুদ্ধিদীপ্ত কলামটিই হলো **Feature Engineering**।

প্রি-প্রসেসিং ডেটাবেজ ম্যানেজমেন্টের দিক থেকে দেখা হয়, আর ফিচার ইঞ্জিনিয়ারিং ডেটা সায়েন্স বা মডেল বিল্ডিংয়ের দিক থেকে দেখা হয়।

Raw Data → Better Features → Better Model

সহজ উদাহরণ:

ধরো তোমার কাছে একটা raw dataset আছে যেখানে শুধু "date of birth" আর "salary" আছে। Feature engineering করে তুমি নতুন ফিচার বানাতে পারো যেমন:

- **age** = current year - date of birth

- **salary_category** = salary কে low/medium/high এ ভাগ করা
- **income_per_year_of_experience** (যদি experience থাকে)

এই নতুন ফিচারগুলো model-কে pattern শেখাতে সাহায্য করে। Raw data থেকে সরাসরি model চালালে accuracy কম হতো, কিন্তু better features দিলে model অনেক স্মার্ট হয়ে যায়।

IMPORTANCE (গুরুত্ব)

Feature engineering না করলে machine learning model যত ভালো অ্যালগরিদমই হোক না কেন, তার performance খারাপ হবে। কারণ:

1. **Improves model accuracy** → সঠিক ফিচার দিলে prediction অনেক accurate হয়।
2. **Helps algorithms learn patterns better** → অ্যালগরিদমগুলো (যেমন Linear Regression, Random Forest) hidden pattern খুঁজে পায় সহজে।
3. **Reduces noise and irrelevant data** → অপ্রয়োজনীয় বা noisy data সরিয়ে দিলে model শুধু important জিনিস শেখে।
4. **Reduces dimensionality** → অনেক ফিচার থাকলে model slow হয় এবং overfit করে। Feature engineering করে ফিচার কমিয়ে দিলে এটা ঠিক হয়।

মনে রাখো একটা কথা:

“Machine learning models are only as good as the features provided.”

অর্থাৎ model যত advanced হোক, যদি ফিচার খারাপ হয় তাহলে outputও খারাপ হবে। Feature engineering হলো data scientist-এর সবচেয়ে creative এবং important কাজ।

FEATURE ENGINEERING-এর চারটা প্রধান ধরন

Slide-এ বলা আছে চারটা উপায়ে feature engineering করা হয়:

Type	Idea (কী করে?)	সহজ বাংলায় ব্যাখ্যা
Feature Transformation (FT)	Modify features	existing ফিচারগুলোকে modify করে ভালো করে (যেমন scaling, missing value fill করা, categorical → numerical)
Feature Construction	Create new features	নতুন ফিচার বানানো (যেমন age বানানো date of birth থেকে)
Feature Selection	Choose important features	শুধু important ফিচারগুলো বেছে নেওয়া, বাকিগুলো বাদ দেওয়া
Feature Extraction	Reduce dimensions	অনেক ফিচার থেকে নতুন কম ফিচার বানানো (যেমন PCA)

একদম বইয়ের মতো বিস্তারিত বোঝা:

- **Feature Transformation (FT):** Raw ফিচারকে শুধু change করা। উদাহরণ: salary ৫০০০০ থেকে ১ লাখ — এটাকে 0-1 এর মধ্যে নিয়ে আসা (scaling) যাতে model biased না হয়।

- **Feature Construction:** Domain knowledge ব্যবহার করে নতুন ফিচার বানানো। উদাহরণ: date থেকে day, month, is_weekend বানানো।
- **Feature Selection:** কোন ফিচারগুলো সত্যি important তা বেছে নেওয়া (correlation, chi-square ইত্যাদি দিয়ে)।
- **Feature Extraction:** অনেক ফিচারকে কমিয়ে নতুন ফিচার বানানো যাতে মূল তথ্য থাকে কিন্তু dimension কমে (PCA, LDA ইত্যাদি)।

DATA TRANSFORMATION

ডেটা ট্রান্সফরমেশন কেন খুব জরুরি?

Data transformation হলো machine learning-এর একটা অত্যন্ত গুরুত্বপূর্ণ step। কারণ raw data-তে বিভিন্ন ফিচারের scale (মানের পরিসর) একদম আলাদা হয়।

উদাহরণ দিয়ে বোঝাই (slide-এর exact example):

ধরো তোমার dataset-এ মানুষের age আর income আছে।

- Age সাধারণত ০ থেকে ১০০ এর মধ্যে হয়। কিন্তু যারা ইনকাম করে তাদের বয়স সাধারণত ২৫ থেকে ৬০ এর মধ্যে।
- Income আবার কয়েক হাজার থেকে কয়েক লাখ টাকা পর্যন্ত হতে পারে।

এখন দেখো — age আর income-এর range-এর মধ্যে অনেক বড় পার্থক্য। যদি তুমি এই raw data সরাসরি model-এ দাও, তাহলে model **biased** হয়ে যাবে। কারণ income-এর বড় সংখ্যাগুলো age-কে ছাপিয়ে যাবে। ফলে model ভালো করে pattern শিখতে পারবে না।

Scaling করলে কী হয়?

- Model-এর accuracy বাড়ে
- Training অনেক faster হয় (কম সময় লাগে)
- অনেকে এটাকেই feature engineering বলে

FEATURE TRANSFORMATION

১. ফিচার ট্রান্সফরমেশন কী?

Feature transformation হলো existing features-কে modify করার প্রক্রিয়া যাতে সেগুলো machine learning algorithm-এর জন্য আরও উপযুক্ত হয়।

এটা মূলত ৪টা কাজ করে:

1. **Handling missing values** → খালি ঘরগুলো সামলানো
2. **Handling categorical values** → categorical data-কে numerical-এ convert করা
3. **Detecting outliers** → অস্বাভাবিক বড়/ছোট মান বের করা
4. **Scaling features to a standard or common range** → সব ফিচারকে একই scale-এ নিয়ে আসা

HANDLING MISSING VALUES

মিসিং ভ্যালু সামলানো

Missing values থাকলে data “crash” করতে পারে এবং model পুরোপুরি নষ্ট হয়ে যেতে পারে। তাই এটাকে অবশ্যই সামলাতে হবে।

দুইটা প্রধান উপায়:

১. Imputation (ফিল করে দেওয়া)

এটা হলো খালি জায়গায় অনুমান করে কিছু ভর্তি করা।

- Mean (গড়) ব্যবহার করা
- Median (মাঝামাঝি মান) ব্যবহার করা
- Mode (সবচেয়ে বেশি যে মান আছে) ব্যবহার করা
- অথবা কোনো logic দিয়ে ভর্তি করা (যেমন: পুরুষ হলে average male salary দিয়ে ভরা)

২. Deletion (মুছে ফেলা)

- যে row বা column-এ missing value আছে সেটা পুরোপুরি delete করে দেওয়া। (তবে এটা তখনই করা উচিত যখন missing value খুব কম। নয়তো অনেক data নষ্ট হয়ে যায়।)

উদাহরণ টেবিল (slide-এর exact table):

User	Device	OS	Transactions
A	Mobile	NA	5
B	Mobile	Android	3
C	NA	IOS	2
D	Tablet	Android	1
E	Mobile	IOS	4

এখানে NA মানে missing value।

- Imputation করে OS-এর NA গুলোতে “Android” বা “IOS” ভরে দিতে পারি
- অথবা C row-টা deletion করে দিতে পারি

ENCODING: HANDLING CATEGORICAL VARIABLE

ক্যাটেগরিক্যাল ভ্যারিয়েবল সামলানো (Encoding)

Data দুই ধরনের হয়:

- **Numerical (Quantitative)** → সংখ্যা (যেমন: age, salary, price)
- **Categorical (Qualitative)** → শ্রেণি (যেমন: color, gender, city, device)

Categorical data আবার দুই ভাগে ভাগ হয়:

- **Nominal** → কোনো order নেই (যেমন: apple, mango, orange)
- **Ordinal** → order আছে (যেমন: low < medium < high)

Encoding কী?

Encoding হলো categorical data-কে numerical format-এ convert করার প্রক্রিয়া। কারণ machine learning algorithm সংখ্যা ছাড়া কাজ করতে পারে না।

এখন পর্যন্ত যা শিখলাম (সারাংশ):

- Data transformation না করলে model biased হয়
- Feature transformation-এর ৪টা মূল কাজ
- Missing value → Imputation বা Deletion
- Categorical data → Encoding দিয়ে numerical বানাতে হয়

ONE-HOT ENCODING

ওয়ান-হট এনকোডিং (সবচেয়ে জনপ্রিয় এনকোডিং)

By far the most common way to represent categorical variables is using **one-hot encoding** (এটাকে one-out-of-N encoding বা dummy variablesও বলে)।

কীভাবে কাজ করে?

- প্রত্যেক unique categorical value-এর জন্য একটা আলাদা column বানানো হয়।
- সেই column-এ শুধু **0** বা **1** (Boolean True/False) বসানো হয়।
- যে row-এ সেই category আছে সেখানে **1**, বাকি সব জায়গায় **0**।

কেন ব্যবহার করি?

Machine learning algorithm শুধু সংখ্যা বোঝে। Categorical data (যেমন: ফলের নাম) সরাসরি দিলে model ভুল করে। One-hot encoding করে সবকিছু numerical বানিয়ে দেয়।

EXAMPLE OF ONE-HOT CODING

উদাহরণ ২ (স্লাইডের **exact example**):

Original table:

Fruit	Categorical value of fruit	Price
apple	1	5
mango	2	10
apple	1	15
orange	3	20

One-Hot Encoded table (get_dummies দিয়ে):

apple	mango	orange	price
1	0	0	5
0	1	0	10
1	0	0	15
0	0	1	20

দেখো, **Fruit** column উধাও হয়ে গেছে। তার বদলে তিনটা নতুন column হয়েছে (apple, mango, orange)। প্রত্যেক row-এ ঠিক একটা ১ আছে, বাকি ০। এটাই dummy variable।

Python-এ কীভাবে করবো?

pd.get_dummies() বা OneHotEncoder() function দিয়ে।

ORDINAL ENCODING

অর্ডিনাল এনকোডিং

Ordinal data মানে এমন categorical data যেখানে **order** বা **rank** আছে (যেমন: low < medium < high, poor < average < good)।

কীভাবে কাজ করে?

- Order অনুসারে সংখ্যা দিয়ে দেয়া হয়।
- Example: low = 1, medium = 2, high = 3

সুবিধা: Order বজায় থাকে, model সেটা বুঝতে পারে।

সতর্কতা: শুধুমাত্র যখন সত্যিকারের order থাকে তখনই ব্যবহার করবে। Nominal data-এ ব্যবহার করলে model ভুল শিখবে।

LABEL ENCODING

লেবেল এনকোডিং (**Target variable-এর জন্য**)

One-hot এবং Ordinal encoder সাধারণত **independent variables (X)**-এর জন্য ব্যবহার হয়।

কিন্তু **target/prediction variable (Y)**-এর জন্য আমরা **Label Encoding** ব্যবহার করি।

কীভাবে কাজ করে?

- প্রত্যেক unique categorical value-কে আলাদা আলাদা সংখ্যা দিয়ে দেয়া হয়।
- কোনো order থাকুক বা না থাকুক, শুধু unique number।

উদাহরণ (স্লাইডের **exact**):

ধরো তোমার dataset-এ **gender** আছে: male / female

- male → 1
- female → 2

আরেকটা উদাহরণ: **ST_NAME** (road name) যদি অনেকগুলো unique রাস্তার নাম হয়, তাহলে প্রত্যেক unique road-কে একটা আলাদা number দিয়ে দেয়া হয়।

গুরুত্বপূর্ণ কথা:

Label encoding শুধু target variable (y)-এর জন্য ব্যবহার করা উচিত। Independent variable-এ ব্যবহার করলে model ভুল ধরে নিতে পারে যে 2 > 1 (order আছে)।

HANDLING OUTLIERS

আউটলায়ার সামলানো

Outliers কী?

Outliers হলো এমন data points যেগুলো বাকি data set থেকে অনেক অনেক আলাদা (খুব বড় বা খুব ছোট)।

কেন সমস্যা?

- Model-এর accuracy নষ্ট করে
- Mean, variance ইত্যাদি গড় মানগুলোকে বেশি প্রভাবিত করে

দুইটা প্রধান উপায়ে **treat** করা হয়:

1. **Trimming** → Outliers-যুক্ত row পুরোপুরি মুছে ফেলা। (যখন outliers খুব কম সংখ্যক হয় তখন ভালো)
2. **Capping** → Outliers-এর জায়গায় অন্য কোনো নির্দিষ্ট মান বসিয়ে দেওয়া (যেমন: upper limit বা lower limit)। (যখন outliers অনেক বেশি হয় তখন এটা ব্যবহার করা হয়)

আউটলায়ার **detect** করার **methods** (স্লাইডে উল্লেখিত):

- z-score
- IQR (Interquartile Range)
- Percentile
- Winsorization

MAPPING FUNCTION

ম্যাপিং ফাংশন

আমরা numerical data-কে categorical-এ বা categorical data-কে numerical-এ convert করতে mapping function ব্যবহার করতে পারি।

উদাহরণ (স্লাইডের **exact**):

Column: **OWN_OCCUPIED**

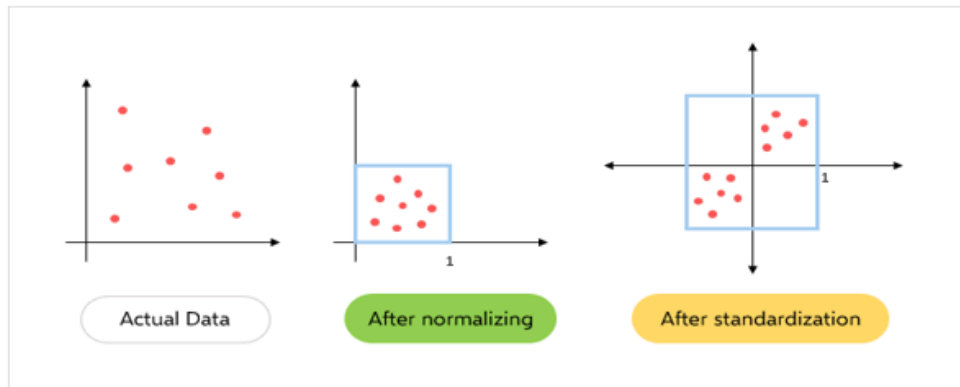
- Y → 1
- N → 2

এভাবে map করে দিলে categorical column numerical হয়ে যায়।

সারাংশ (এই অংশ থেকে যা শিখিলে):

- One-hot encoding → nominal categorical data-এর জন্য best (dummy columns বানায়)
- Ordinal encoding → যেখানে order আছে সেখানে
- Label encoding → শুধু target variable (y)-এর জন্য
- Outliers → trimming বা capping দিয়ে সামলাতে হয়
- Mapping function → সহজ conversion-এর জন্য

FeatureScaling



Feature scaling is an essential step in the machine-learning process. By scaling the features, we can help improve your model's performance and ensure that all features are given a fair chance.

FEATURE SCALING

ফিচার স্কেলিং কী?

Feature scaling হলো dataset-এর সব ফিচারকে একই **common scale**-এ নিয়ে আসার প্রক্রিয়া। এটা না করলে কিছু ফিচার (যেমন income যার মান লাখ লাখ) অন্য ফিচারকে (যেমন age যার মান ০-১০০) পুরোপুরি dominate করে ফেলে। ফলে model biased হয় এবং ভালো করে শিখতে পারে না।

কেন এত গুরুত্বপূর্ণ?

- Model-এর performance অনেক বাড়ায়
- সব ফিচারকে **fair chance** দেয়
- Training অনেক faster হয়
- Distance-based algorithm-এ (SVM, KNN, Clustering) একদম must

দুইটা মেইন টাইপ:

i. **Standardization**

ii. **Normalization**

STANDARDIZATION

স্ট্যান্ডার্ডাইজেশন (**Z-score scaling**)

Standardization করে প্রত্যেক ফিচারের **mean = 0** এবং **standard deviation = 1** হয়। এটা Gaussian (Normal) distribution-এর data-এর জন্য সবচেয়ে ভালো কাজ করে (যেমন Linear Regression, Logistic Regression)।

ফর্মুলা:

$$y = \frac{x - \mu}{\sigma}$$

যেখানে,

- x = original value
- μ = mean
- σ = standard deviation

উদাহরণ (স্লাইডের **exact**):

ধরো mean = 10.0 এবং standard deviation ≈ 5.0

প্রথম value = 20.7

$$y = 20.7 - 10 = 10.75 = 2.14 \times 5 = 10.7$$

বিশেষ সুবিধা:

- Mean ও standard deviation নতুন data-এর জন্যও robust (মিন-ম্যাক্সের চেয়ে ভালো)
- Outlier থাকলেও তুলনামূলকভাবে ভালো কাজ করে

NORMALIZATION

নরমালাইজেশন

Normalization হলো data-কে একটা নির্দিষ্ট range-এ (সাধারণত 0 থেকে 1) নিয়ে আসা। এটা data pre-processing-এর একটা basic step। মূল উদ্দেশ্য হলো duplicated data কমানো এবং সব ফিচারকে সমান গুরুত্ব দেওয়া।

NORMALIZATION-এর TECHNIQUE গুলো:

- Min-Max Normalization
- Robust Scaling Normalization
- Z-score Normalization
- Decimal Scale Normalization
- Log Scale Normalization

MIN-MAX NORMALIZATION

মিন-ম্যাক্স নরমালাইজেশন (সবচেয়ে সাধারণ)

এটাকে সাধারণত শুধু **Normalization** বলা হয়। ফিচারকে 0 থেকে 1 এর মধ্যে নিয়ে আসে।

উদাহরণ (স্লাইডের **exact salary table**):

Salary

64000

55000

19000

100000

75000

Max = 100000

Min = 19000

ফর্মুলা:

$$X_{\text{normalized}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

উদাহরণ হিসাব:

- 64000 এর জন্য:
 $64000 - 19000 = 45000$
 $\frac{45000}{81000} \approx 0.555$
- 55000 এর জন্য:
 $55000 - 19000 = 36000$
 $\frac{36000}{81000} \approx 0.444$

কখন **Min-Max** ভালো কাজ করে?

- Dataset-এর upper এবং lower bound আগে থেকে জানা থাকলে
- খুব কম বা কোনো outlier না থাকলে
- Data distribution unknown বা non-Gaussian হলে এবং uniform হলে
- Distribution-এর original shape বজায় রাখতে চাইলে

ROBUST SCALING NORMALIZATION

রোবাস্ট স্কেলিং নরমালাইজেশন

এটা **outlier**-এর বিরুদ্ধে খুব শক্তিশালী। Median এবং IQR (Interquartile Range) ব্যবহার করে।

ফর্মুলা:

$$X_{\text{scaled}} = \frac{X - \text{median}}{IQR} \quad X_{\text{scaled}} = \frac{IQR}{X - \text{median}}$$

(যেখানে $IQR = Q3 - Q1$)

উদাহরণ (স্লাইডের **exact**):

Original value = 30

Input's median = 25

$IQR = 20$

$$\text{Robust scaled value} = \frac{30 - 25}{20} = \frac{5}{20} = 0.25$$

(স্লাইডে 28 লেখা আছে, কিন্তু হিসাব অনুসারে median 25 হলে 0.25 হয় — তবে মূল কথা একই)

কেন **Robust Scaling** ব্যবহার করবো?

- Outlier থাকলে standard scaler বা min-max ভেঙে যায়
- Skewed বা anomalous data-এর জন্য সবচেয়ে ভালো
- Scaled value-এর median = 0 এবং IQR = 1 হয়

সারাংশ (এই অংশ থেকে যা শিখিলে):

- Feature scaling ছাড়া distance-based model (KNN, SVM) খারাপ করে
- Standardization $\rightarrow$ mean=0, std=1 (Gaussian data-এ best)
- Normalization $\rightarrow$ 0 থেকে 1 এর মধ্যে (Min-Max সবচেয়ে সাধারণ)
- Robust Scaling $\rightarrow$ outlier থাকলে এটাই ব্যবহার করবে

Z-SCORE NORMALIZATION

জেড-স্কোর নরমালাইজেশন (এটাই আসলে **Standardization**)

Z-score normalization হলো Standardization-এরই আরেক নাম। এটা প্রত্যেক ফিচারকে এমনভাবে স্কেল করে যাতে **mean = 0** এবং **standard deviation = 1** হয়।

ফর্মুলা:

$$Z = \frac{x - \mu}{\sigma}$$

(যেখানে x = original value, μ = mean, σ = standard deviation)

এটা Gaussian/normal distribution-এর data-এর জন্য সবচেয়ে ভালো কাজ করে। আগের অংশে আমরা Standardization হিসেবে এটাই দেখেছি।

DECIMAL SCALE NORMALIZATION

ডেসিমাল স্কেল নরমালাইজেশন

Decimal scaling normalization ফিচারের value-গুলোকে ১০-এর পাওয়ার দিয়ে স্কেল করে যাতে প্রত্যেক ফিচারের সবচেয়ে বড় absolute value ১-এর চেয়ে ছোট হয়ে যায়।

কখন ব্যবহার করবো?

- Dataset-এর range আগে থেকে জানা থাকলে
- বিভিন্ন ফিচারের range একদম আলাদা আলাদা হলে (যেমন CGPA 0-4, salary ১০ হাজার-১ লাখ, money ১০০-১০ লাখ)
- Absolute magnitude (কত বড় সংখ্যা) গুরুত্বপূর্ণ হলে

ফর্মুলা:

$$X_{decimal} = \frac{X}{10^d}$$

যেখানে,

- X = original feature value
- d = সবচেয়ে ছোট integer যাতে সবচেয়ে বড় absolute value < 1 হয়

উদাহরণ (স্লাইডের **exact**):

- সবচেয়ে বড় value = 350 $\rightarrow d=3 \rightarrow$ divide by $10^3=1000$

ডেসিমাল স্কেল নরমালাইজেশনের টেবিল (স্লাইডের **exact**):

CGPA	Formula	After Decimal Normalization
2	$2/10$	0.2
3	$3/10$	0.3

Money	Formula	After Decimal Normalization
500	$500/1000$	0.5
320	$320/1000$	0.32

Salary	Formula	After Decimal Normalization
32000	$32000/100000$	0.32
28000	$28000/100000$	0.28

LOG SCALING NORMALIZATION

লগ স্কেলিং নরমালাইজেশন

Log scaling normalization প্রত্যেক data point-এর log নিয়ে data-কে logarithmic scale-এ নিয়ে আসে।

কখন ব্যবহার করবো?

- Data যখন **several orders of magnitude** এর হয় (যেমন ১০, ১০০, ১০০০, ১০ লাখ)
- Exponential growth বা decay pattern থাকলে (যেমন: জনসংখ্যা বৃদ্ধি, company revenue বৃদ্ধি)
- Scale compress করে model-কে pattern শেখানো সহজ করে

ফর্মুলা:

$$X_{\log} = \log(X) \quad X_{\log} = \log(X)$$

(সাধারণত log base 10 বা natural log ব্যবহার হয়)

উদাহরণ (স্লাইডের **exact**):

$$X = 200 \rightarrow X_{\log} = \log(200) \approx 2.30103 \quad X_{\log} = \log(200) \approx 2.30103$$

লগ স্কেলিং টেবিল (স্লাইডের **exact**):

Value	$\log_{10}(\text{Value})$
10	1
100	2
1000	3
100000	5



Comparison: Normalization vs Standardization

ডেটা স্কেলিং-এর এই দুটি পদ্ধতিই মেশিন লার্নিং মডেলে ডেটার রেঞ্জ ঠিক করার জন্য ব্যবহৃত হয়, কিন্তু এদের কাজ করার ধরন আলাদা।

বৈশিষ্ট্য (Feature)	Normalization (Min-Max Scaling)	Standardization (Z-Score Normalization)

মূল লক্ষ্য	ডেটাকে একটি নির্দিষ্ট রেঞ্জের (যেমন: ০ থেকে ১) মধ্যে নিয়ে আসা।	ডেটাকে এমনভাবে রূপান্তর করা যাতে Mean = ০ এবং Std. Deviation = ১ হয়।
আউটলায়ার (Outliers)	আউটলায়ারের প্রতি খুবই সেনসিটিভ। একটি বড় সংখ্যা পুরো স্কেল নষ্ট করে দিতে পারে।	আউটলায়ারের প্রতি কম সেনসিটিভ কারণ এটি গড় (mean) ব্যবহার করে।
ডেটা ডিস্ট্রিবিউশন	ডেটার ডিস্ট্রিবিউশন কেমন তা নিয়ে কোনো পূর্বশর্ত নেই।	ধরে নেওয়া হয় ডেটা Normal Distribution বা তার কাছাকাছি।
ব্যবহারযোগ্যতা	যেখানে ডেটার আসল রেঞ্জ এবং রিলেশন বজায় রাখা জরুরি।	যেখানে অ্যালগরিদম ধরে নেয় ডেটা নরমালি ডিস্ট্রিবিউটেড।
ইন্টারপ্রিটেশন	মূল মানের রিলেশন বজায় থাকে বলে বোঝা সহজ।	স্কেল এবং ইউনিট বদলে যায় বলে ইন্টারপ্রিটেশন কিছুটা কঠিন।
অ্যালগরিদম উদাহরণ	K-Nearest Neighbors (KNN), Neural Networks.	Linear Regression, Logistic Regression, SVM.

🤔 Which Procedure is Appropriate? (কখন কোনটি ব্যবহার করবে?)

কোনটা ব্যবহার করবে তা পুরোপুরি নির্ভর করে তোমার প্রবলেম টাইপ এবং তুমি কোন অ্যালগরিদম ব্যবহার করছো তার ওপর।

১. ডিস্ট্যান্স-ভিত্তিক অ্যালগরিদমে গুরুত্ব (Distance-based Algorithms):

- যদি তুমি **SVM**, **KNN**, বা **Clustering**-এর মতো অ্যালগরিদম ব্যবহার করো যেখানে 'দূরত্ব' মেপে কাজ করা হয়, সেখানে ডেটা স্কেলিং করা বাধ্যতামূলক। স্কেলিং না করলে বড় ভ্যালুগুলো (যেমন: Income) ছোট ভ্যালুগুলোকে (যেমন: Age) ডমিনেট করবে।

২. নন-ডিস্ট্যান্স অ্যালগরিদমে গুরুত্ব:

- Naive-Bayes** বা **Tree-based algorithms** (যেমন: Decision Tree, Random Forest)-এ স্কেলিং খুব একটা প্রভাব ফেলে না। এগুলোতে স্কেলিং ছাড়াই ভালো রেজাল্ট পাওয়া যায়।

৩. কখন Normalization বেছে নেবে?

- যখন তোমার ডেটাসেটের ফিচারগুলোর মধ্যে রেঞ্জের অনেক বড় গ্যাপ থাকে এবং আউটলায়ার খুব একটা নেই।
- বিশেষ করে **Image Processing** এবং **Neural Networks**-এর ক্ষেত্রে এটি বেশি কার্যকর।

৪. কখন **Standardization** বেছে নেবে?

- যদি তোমার ডেটাসেটে আউটলায়ার (**Outliers**) থাকে, তবে চেষ্টা বন্ধ করে **Standardization** ব্যবহার করা ভালো।
- বেশিরভাগ স্ট্যাটিস্টিক্যাল অ্যালগরিদম (Linear/Logistic Regression) এবং SVM-এ এটি ভালো কাজ করে।

সারসংক্ষেপ (**Final Verdict**):

বেশিরভাগ সাধারণ ক্ষেত্রে এবং প্রোডাকশন লেভেলে **Standardization** সবথেকে ভালো রেজাল্ট দেয় (Robust)। তবে ডিপ লার্নিং বা ইমেজের ক্ষেত্রে Normalization অগ্রাধিকার পায়।

WHICH PROCEDURE IS APPROPRIATE?

কোনটা কখন ব্যবহার করবো?

- **Distance-based algorithm** (SVM, KNN, Clustering) → **Scaling** খুব জরুরি
- **Tree-based / Naive Bayes** → **Scaling**-এর দরকার কম
- অনেক বড় range difference থাকলে → **Normalization**
- Outlier থাকলে → **Standardization / Robust Scaling**
- বেশিরভাগ ক্ষেত্রে → **Standardization** সবচেয়ে ভালো কাজ করে

সারাংশ (এই অংশ থেকে মূল কথা):

- Z-score = Standardization (mean 0, std 1)
- Decimal Scale → 10^d দিয়ে ছোট করা
- Log Scaling → exponential data-এর জন্য best
- Normalization vs Standardization-এর তুলনা



Topic: Feature Construction

১. Feature Construction কী? (The Concept)

বিদ্যমান (Existing) ফিচারগুলো ব্যবহার করে অথবা তোমার ডোমেইন নলেজ (Domain Knowledge) কাজে লাগিয়ে একদম নতুন কোনো ফিচার তৈরি করার প্রক্রিয়াকেই **Feature Construction** বলে।

- মূল উদ্দেশ্য: ডেটার ভেতরে লুকিয়ে থাকা সম্পর্কগুলো (Hidden relationships) মডেলের সামনে তুলে ধরা এবং প্রেডিকশন পাওয়ার বাড়ানো।

২. কেন এটি প্রয়োজন? (The "Why")

অনেক সময় Raw data সরাসরি মডেলে দিলে মডেল প্যাটার্ন বুঝতে পারে না। যেমন: জন্ম তারিখ (Date of Birth) সরাসরি মডেলে দিলে লাভ নেই, কিন্তু তা থেকে যদি 'বয়স' (Age) বের করে দাও, তবে মডেল খুব সহজে প্যাটার্ন ধরতে পারবে।

৩. Feature Construction করার ৩টি প্রধান উপায় (Techniques with Examples):

A. Repurposing Existing Features (remixing old songs 🎵):

পুরানো ফিচারগুলোকে গাণিতিক ভাবে যোগ, বিয়োগ, গুণ বা ভাগ করে নতুন ফিচার তৈরি করা।

- **Example 1:** তোমার কাছে 'বাবার বয়স' এবং 'মায়ের বয়স' আছে। তুমি এই দুটি যোগ করে ২ দিয়ে ভাগ করে '**Average Parent's Age**' নামে একটি নতুন ফিচার তৈরি করলে।
- **Example 2 (Unit Conversion):** উচ্চতা 'cm'-এ আছে, তুমি সেটাকে 'inches'-এ কনভার্ট করলে যদি তোমার মডেলের জন্য সেটা বেশি সুবিধাজনক হয়।

- **Example 3 (Business):** 'Revenue' (মোট আয়) এবং 'Total Customers' আছে। তুমি এই দুটি ভাগ করে 'Revenue per Customer' বের করলে, যা ব্যবসার উন্নতির আসল চিত্র দেখায়।

B. Using Domain Expertise (Consulting a chef 🍳):

তুমি যে বিষয়ে কাজ করছো (যেমন: ডাক্তারী, ব্যাংকিং বা ই-কমার্স), সেই বিষয়ের বাস্তব অভিজ্ঞতা কাজে লাগিয়ে ফিচার তৈরি করা।

- **Example 1 (Customer Churn):** টেলিকম কোম্পানি থেকে কাস্টমার চলে যাবে কিনা তা প্রেডিক্ট করতে তুমি 'Number of months since last purchase' ফিচারটি যোগ করলে। কারণ তুমি জানো যে অনেকদিন কেনাকাটা না করলে কাস্টমার চলে যাওয়ার সম্ভাবনা বেশি।
- **Example 2 (Time-Series):** তোমার কাছে শুধু 'Date' (যেমন: 2023-12-25) আছে। ডোমেইন নলেজ থেকে তুমি জানো ছুটির দিনে কেনাকাটা বেশি হয়। তাই তুমি নতুন ফিচার বানালে— 'Is_Weekend' (Yes/No) অথবা 'Season' (Winter/Summer)।

C. Using Feature Selection/Extraction Algorithms (Personal Shopper 🛍️):

অ্যালগরিদমের সাহায্য নিয়ে দেখা কোন ফিচারগুলো সবচেয়ে বেশি শক্তিশালী এবং সেই ভিত্তিতে নতুন ডাইমেনশন তৈরি করা।

- **Example: PCA (Principal Component Analysis)** ব্যবহার করে অনেকগুলো ফিচার থেকে প্রধান বৈশিষ্ট্যগুলো নিয়ে নতুন এবং অল্প সংখ্যক ফিচার (Principal Components) তৈরি করা। এটি জটিল ডেটাকে ছোট কিন্তু শক্তিশালী ফিচারে রূপান্তর করে।

 Summary Table for Quick Revision

পদ্ধতি	মূল কাজ	উদাহরণ
Repurposing	বিদ্যমান ফিচারের ম্যাথমেটিকাল কম্বিনেশন	Revenue / Customers = Revenue_per_head
Domain Knowledge	বাস্তব অভিজ্ঞতা থেকে নতুন তথ্য বের করা	Date → Weekend / Holiday
Algorithms	গাণিতিক মডেল (যেমন PCA) ব্যবহার	100 features → 5 Principal Components

 Topic: Feature Extraction & Dimensionality Reduction

১. Curse of Dimensionality (ডাইমেনশনালিটির অভিশাপ):

মেশিন লার্নিংয়ে ফিচারের সংখ্যা যখন অনেক বেশি হয়ে যায়, তখন মডেলের পক্ষে আসল প্যাটার্ন খুঁজে পাওয়া কঠিন হয়ে পড়ে। একেই বলে 'Curse of Dimensionality'।

- সমস্যা: ডেটা যত বেশি জটিল (High dimensional) হয়, মডেলের ট্রেনিং তত স্লো হয় এবং accuracy কমে যাওয়ার ঝুঁকি থাকে (Overfitting)।
- সমাধান: Dimensionality Reduction (ডাইমেনশন কমানো)।

২. Dimensionality Reduction-এর দুটি পথ:

ডেটার ডাইমেনশন কমানোর প্রধানত দুটি উপায় আছে:

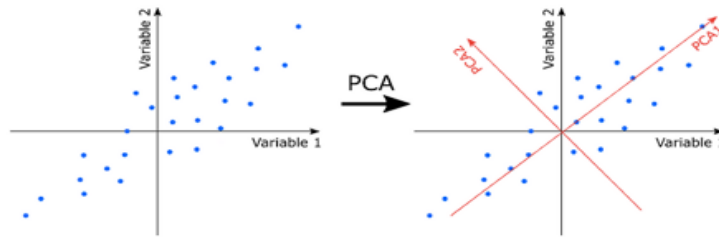
1. **Feature Selection:** আসল ফিচারগুলো থেকে সেরা কয়েকটি বেছে নেওয়া (এখানে ফিচারের কোনো পরিবর্তন হয় না)।
2. **Feature Extraction:** অনেকগুলো ফিচারের তথ্যকে সংকুচিত করে একদম নতুন এবং অল্প কিছু ফিচার তৈরি করা (এখানে ফিচারের অরিজিনাল রূপ বদলে যায়)।

৩. Feature Extraction-এর প্রধান টেকনিকসমূহ (Methods):

A. Principal Component Analysis (PCA)

এটি একটি **Unsupervised Technique** (Unsupervised Technique হলো মেশিন লার্নিংয়ের এমন একটি পদ্ধতি যেখানে অ্যালগরিদমকে এমন ডেটা দেওয়া হয় যার কোনো **Label** বা **Target Output** থাকে না। অর্থাৎ, ডেটাতে কোনো "সঠিক উত্তর" দেওয়া থাকে না, কম্পিউটারকে নিজের বুদ্ধিতে ডেটার ভেতরকার লুকানো প্যাটার্ন (**Hidden Patterns**) বা স্ট্রাকচার খুঁজে বের করতে হয়।)। এটি ডেটার ভেতরের সবচেয়ে গুরুত্বপূর্ণ 'Variance' বা পরিবর্তনগুলো খুঁজে বের করে এবং সেগুলোকে **Principal Components (PC)** হিসেবে প্রকাশ করে।

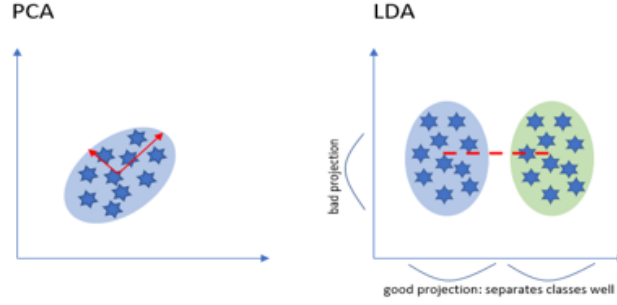
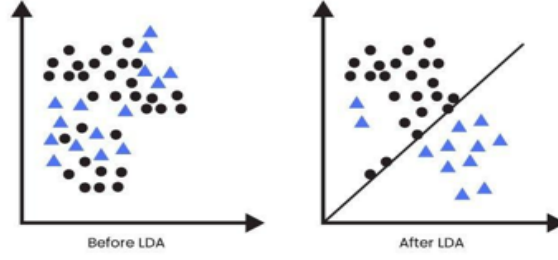
- কিভাবে কাজ করে? ধরো তোমার কাছে ১০০টি ফিচার আছে। PCA সেগুলোকে এমনভাবে কম্বাইন করবে যাতে মাত্র ৫-১০টি নতুন ফিচারের (PC1, PC2...) মাধ্যমেই পুরো ডেটার ৯০% তথ্য প্রকাশ করা যায়।
- মূল বৈশিষ্ট্য: এটি ফিচারের অরিজিনাল রিপ্রেজেন্টেশন বদলে ফেলে (যেমন: 'উচ্চতা' ও 'ওজন' মিলে নতুন একটি ভ্যালু হয়ে যাবে যা চেনা যাবে না)।



B. LDA (Linear Discriminant Analysis)

LDA একটি **Supervised** মেশিন লার্নিং অ্যালগরিদম। (**Supervised:** শিক্ষক ছাত্রকে বলে দিচ্ছেন এটা 'আপেল' ওটা 'কমলা' (**Label** দেওয়া আছে)। **Unsupervised:** ছাত্রের সামনে এক ঝুড়ি ফল রাখা হলো কোনো নাম ছাড়া। ছাত্র ফলের আকার, রঙ বা স্বাদ দেখে নিজে নিজে গ্রুপ বানাতে (**Label** নেই, শুধু প্যাটার্ন আছে)। এটি মূলত ক্লাসিফিকেশন প্রবলেমে ব্যবহৃত হয়। এর প্রধান কাজ হলো এমন একটি দিক (Direction) খুঁজে বের করা যা বিভিন্ন ক্যাটাগরি বা ক্লাসের মধ্যে পার্থক্য বা দূরত্ব সর্বোচ্চ করে।

- কিভাবে কাজ করে? ধরো তোমার কাছে দুই ধরনের ডেটা আছে (যেমন: 'সুস্থ' এবং 'অসুস্থ')। LDA ডেটার ডাইমেনশন এমনভাবে কমাবে যাতে এই দুই গ্রুপের মধ্যে গ্যাপ সবচেয়ে বেশি থাকে এবং গ্রুপের ভেতরের ডেটাগুলো কাছাকাছি থাকে।
- উদ্দেশ্য: Classification-এর পারফরম্যান্স বাড়ানো।
- উদাহরণ: তোমার কাছে কয়েকশ ফিচার আছে যার মাধ্যমে তুমি 'Fraud' এবং 'Non-fraud' ট্রানজাকশন আলাদা করতে চাও। LDA এই অনেকগুলো ফিচারের তথ্যকে মাত্র ১টি বা ২টি নতুন ফিচারে নিয়ে আসবে যা এই দুই গ্রুপকে একদম পরিষ্কারভাবে আলাদা (Separate) করতে পারে।



C. T-SNE (T-distributed Stochastic Neighbor Embedding)

T-SNE একটি **Non-linear** ডাইমেনশনালিটি রিডাকশন টেকনিক। এটি মূলত হাই-ডাইমেনশনাল ডেটাকে ২ডি (2D) বা ৩ডি (3D)-তে নামিয়ে এনে **Visualization** করার জন্য ব্যবহৃত হয়।

- কিভাবে কাজ করে? এটি ডেটার 'Local Structure' বজায় রাখে। অর্থাৎ, হাই-ডাইমেনশন স্পেসে যে ডেটাগুলো একে অপরের কাছে ছিল, লো-ডাইমেনশনে আসার পরেও তারা কাছাকাছিই থাকবে। এটি এমন সব ডেটাকে আলাদা করতে পারে যা একটি সোজা দাগ (Linear) দিয়ে আলাদা করা সম্ভব নয়।
- উদ্দেশ্য: ডেটার ভেতরে কোথায় কোথায় 'ক্লাস্টার' বা গ্রুপ আছে তা মানুষের চোখের উপযোগী করে দেখা।
- উদাহরণ: তোমার কাছে হাজার হাজার হাতে লেখা সংখ্যার (MNIST Dataset) ছবি আছে। T-SNE ব্যবহার করলে তুমি দেখবে যে '0', '1', '2'—এই সংখ্যাগুলো আলাদা আলাদা সুন্দর গুচ্ছ বা ক্লাস্টার আকারে ২ডি ম্যাপে দেখা যাচ্ছে।

T-SNE (t-distributed Stochastic Neighbor Embedding) শুনতে একটু কঠিন মনে হলেও এর আসল কাজটা খুবই মজার। চলো একদম সহজভাবে ভেঙে দেখি।

১. মূল কনসেপ্ট (The Basic Idea)

ধরো তোমার কাছে একটি ৩ডি (3D) বল আছে যার ভেতরে অনেকগুলো বিন্দু ছড়ানো। এখন তুমি যদি এই বলটাকে হাত দিয়ে চেপে একদম ২ডি (2D) কাগজের মতো চ্যাপ্টা করে ফেলো, তাহলে বিন্দুগুলোর মধ্যে দূরত্ব উলটপালট হয়ে যেতে পারে।

T-SNE-র কাজ হলো: চ্যাপ্টা বা ডাইমেনশন কমানোর সময় খেয়াল রাখা যেন হাই-ডাইমেনশনে যে বিন্দুগুলো একে অপরের কাছে (**Neighbor**) ছিল, লো-ডাইমেনশনেও তারা যেন কাছাকাছিই থাকে।

- **Stochastic:** এর মানে হলো এতে কিছুটা র‍্যান্ডম বা দৈব চয়ন আছে।
- **Neighbor Embedding:** এর মানে হলো পাড়া-প্রতিবেশীদের (কাছাকাছি ডেটা পয়েন্ট) সম্পর্ক বজায় রাখা।

২. এটি কিভাবে কাজ করে? (Step-by-Step)

1. **High-Dimensional Space:** প্রথমে এটি হাই-ডাইমেনশনে প্রতিটি ডেটা পয়েন্টের জন্য একটি 'Probability' বের করে যে, কোন পয়েন্ট কার কত কাছে আছে।
2. **Low-Dimensional Space:** এরপর এটি ২ডি বা ৩ডি স্পেসে বিন্দুগুলোকে র‍্যান্ডমলি বসায়।
3. **Optimization:** এরপর এটি বিন্দুগুলোকে এমনভাবে নাড়াচাড়া (Shift) করতে থাকে যাতে হাই-ডাইমেনশনের সেই 'কাছাকাছি থাকার সম্পর্ক' লো-ডাইমেনশনেও মিলে যায়।

একটা মজার উদাহরণ: > ধরো তোমার ক্লাসের সব স্টুডেন্টের পছন্দ-অপছন্দ নিয়ে একটা বিশাল ডেটাসেট আছে (১০০টা প্রশ্ন: কার বিরিয়ানি পছন্দ, কার ফুটবল পছন্দ ইত্যাদি)। এখন এই ১০০টা ফিচারের ডেটা চোখে দেখে বোঝা সম্ভব না কে কার মতো। তুমি যদি **T-SNE** চালাও, সে দেখবে যারা ফুটবল ও বিরিয়ানি পছন্দ করে তারা হাই-ডাইমেনশনে একে অপরের কাছে আছে। তখন সে ২ডি গ্রাফে ওই স্টুডেন্টদের একটা আলাদা '**Cluster**' বা গুচ্ছ বানিয়ে দেবে। তুমি গ্রাফ দেখলেই বুঝবে— "আচ্ছা, এই কোণার গুচ্ছটা হলো বিরিয়ানি লাভার গ্রুপ!"

৩. T-SNE বনাম PCA (কেন T-SNE স্পেশাল?)

- **PCA (Linear):** এ শুধু সোজা লাইন ধরে ডেটা আলাদা করতে পারে। যদি ডেটা জিলাপির মতো পেঁচানো হয় (Non-linear), PCA সেটা ঠিকমতো খুলতে পারে না।
- **T-SNE (Non-linear):** এ জিলাপির প্যাঁচ খুলে ভেতরের ডেটা পয়েন্টগুলোর আসল সম্পর্ক বের করে আনতে পারে।

🔍 Topic: Feature Selection (Filter Methods)

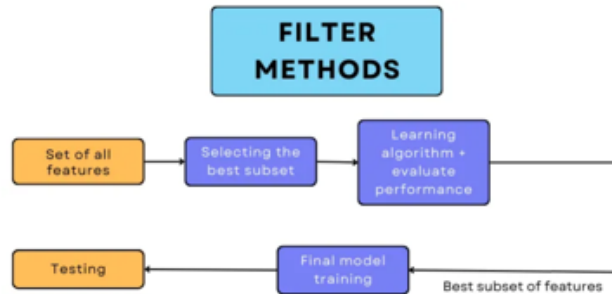
১. Feature Selection কী?

ডেটাসেটের শত শত ফিচারের মধ্যে সব ফিচার প্রেডিকশনের জন্য দরকার হয় না। অপ্রয়োজনীয় বা নয়েজ তৈরি করে এমন ফিচার বাদ দিয়ে শুধু সবচেয়ে গুরুত্বপূর্ণ ফিচারগুলো বেছে নেওয়ার প্রক্রিয়াকেই **Feature Selection** বলে।

- মূল মন্ত্র: Less is More (কম ফিচারে ভালো মডেল)।

1. Filter Methods (ফিল্টার মেথড):

এটি সম্পূর্ণ **Statistical Measure** বা গাণিতিক হিসাবের ওপর ভিত্তি করে কাজ করে। এর সবচেয়ে বড় বৈশিষ্ট্য হলো— এটি কোনো মেশিন লার্নিং অ্যালগরিদম ব্যবহার করে না (Not based on a learning algorithm)। এটি মডেল ট্রেনিংয়ের আগেই ডেটা থেকে ফিচার ফিল্টার করে ফেলে।



🔧 Filter Method-এর বিভিন্ন উপায় (Techniques):

A. Correlation (সহসম্পর্ক):

এটি চেক করে একটি ফিচারের পরিবর্তনের সাথে Target Variable (যেটা প্রেডিক্ট করতে হবে) কতটা পরিবর্তিত হয়।

- **Concept:** যদি কোনো ফিচারের সাথে টার্গেটের গভীর সম্পর্ক থাকে (High Correlation), তবে সেটি রাখা হয়। সম্পর্ক খুব কম হলে সেটি বাদ দেওয়া হয়।
- **Method:** পাইথনে সাধারণত `corr()` ফাংশন দিয়ে এটি চেক করা হয়।

B. Variance Threshold (ভ্যারিয়েন্স থ্রেশহোল্ড):

- **Concept:** যদি কোনো ফিচারের সব ভ্যালু প্রায় একই হয় (Low Variance), তবে সেই ফিচার মডেলকে নতুন কিছু শেখাতে পারে না।
- উদাহরণ: ধরো 'দেশ' কলামের সবার মানই 'Bangladesh'। এখানে কোনো ভ্যারিয়েশন নেই, তাই এই ফিচারটি কোনো কাজের না। এটি 'কিক আউট' বা বাদ দেওয়া হয়।

C. Chi-Squared Test (চি-স্কয়ার টেস্ট)

এটি মূলত **Categorical Input** এবং **Categorical Output**-এর মধ্যে সম্পর্ক খোঁজার জন্য ব্যবহার করা হয়। এটি অনেকটা গোল্ডম্যানের মতো কাজ করে, দেখে যে দুটি ক্যাটাগরির মধ্যে কোনো "গোপন যোগসূত্র" আছে কি না।

- সহজ উদাহরণ: ধরো, তুমি টাইটানিক জাহাজের ডেটাসেট নিয়ে কাজ করছো। তোমার কাছে ফিচার আছে **Gender** (Male/Female) এবং টার্গেট ভ্যারিয়েবল হলো **Survived** (Yes/No)। এখন Chi-Square টেস্ট দেখবে যে "লিঙ্গ" কি "বেঁচে থাকার" ওপর প্রভাব ফেলেছে? দেখা গেল মহিলারা বেশি বেঁচেছে। তার মানে Gender এবং Survived-এর মধ্যে একটা শক্তিশালী সম্পর্ক আছে। তাই Gender ফিচারটি মডেলে রাখা হবে।
- কখন ব্যবহার করবে: যখন ইনপুট এবং আউটপুট উভয়ই টেক্সট বা ক্যাটাগরি আকারে থাকে (যেমন: রং, শহর, লিঙ্গ, গ্রেড)।

D. ANOVA (Analysis of Variance)

ANOVA বিচারকের মতো কাজ করে। এটি ব্যবহার করা হয় যখন তোমার **Numerical Input** থাকে কিন্তু **Categorical Output** থাকে। এটি বিভিন্ন গ্রুপের 'গড়' (Mean) চেক করে দেখে তারা আসলে কতটা আলাদা।

- সহজ উদাহরণ: ধরো তোমার কাছে ডেটা আছে— বিভিন্ন মানুষের **Income** (Numerical) এবং তারা কোন **City**-তে (Categorical) থাকে। তুমি চেক করতে চাও "শহর ভেদে মানুষের ইনকামে কি আসলেই কোনো পার্থক্য আছে?" ANOVA টেস্ট করে যদি দেখা যায় ঢাকা, চট্টগ্রাম আর সিলেটের মানুষের গড় ইনকামে অনেক বড় পার্থক্য আছে, তবে এর মানে হলো 'City' ফিচারটি 'Income' প্রেডিক্ট করার জন্য খুবই গুরুত্বপূর্ণ। কিন্তু যদি দেখা যায় সব শহরের ইনকাম প্রায় সমান, তবে 'City' ফিচারটি মডেলে রাখার কোনো দরকার নেই।
- কখন ব্যবহার করবে: যখন তোমার ইনপুট সংখ্যা (যেমন: বয়স, উচ্চতা, বেতন) কিন্তু আউটপুট বা টার্গেট হলো ক্যাটাগরি (যেমন: রেজাল্ট পাস/ফেল, রোগের ধরন)।
-

💡 Filter Method-এর সুবিধা ও অসুবিধা

সুবিধা (Pros)	অসুবিধা (Cons)
এটি অনেক দ্রুত (Computationally fast)	এটি ফিচারের মধ্যকার পারস্পরিক সম্পর্ক (Inter-dependence) দেখে না।
মডেল ওভারফিটিং হওয়ার সম্ভাবনা কমায়।	মডেলের পারফরম্যান্সের সাথে এর সরাসরি কোনো সম্পর্ক নেই।
ছোট বা বড় সব ডেটাসেটেই চলে।	অনেক সময় দরকারী ফিচারও বাদ পড়ে যেতে পারে যদি স্ট্যাটিস্টিক্স না মেলে।

📝 পরীক্ষার জন্য শর্টকাট টেবিল (Quick Revision)

Method Name	Input Type	Output (Target) Type	Main Focus

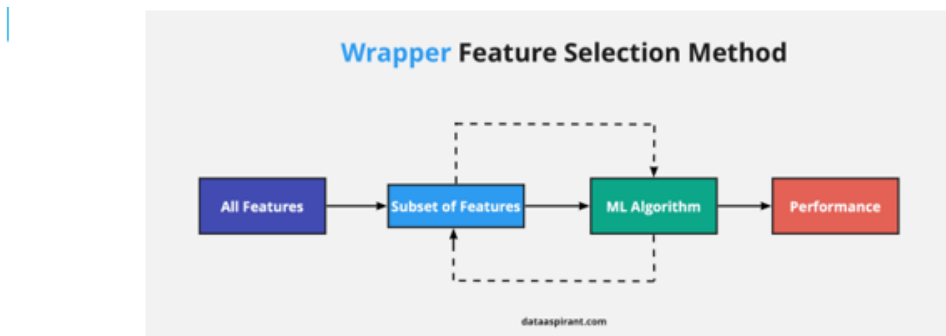
Correlation	Numerical	Numerical	Linear relationship
Chi-Squared	Categorical	Categorical	Independence/Association
ANOVA	Numerical	Categorical	Difference in Means
Variance Threshold	Any	N/A	

Topic: Feature Selection (Wrapper Methods)

2. Wrapper Methods কী? (The Dating App Analogy 📱)

স্লাইডের ভাষায়, এটি ফিচারের জন্য একটি "Dating App"-এর মতো। ফিল্টার মেথড শুধু স্ট্যাটিস্টিক্স দেখে, কিন্তু **Wrapper Method** বিভিন্ন ফিচারের কম্বিনেশন নিয়ে বার বার মেশিন লার্নিং অ্যালগরিদম চালিয়ে দেখে কোন জোড়া বা গ্রুপটি সবচেয়ে ভালো রেজাল্ট (Accuracy) দিচ্ছে।

- মূল পার্থক্য: এটি ফিচারের মধ্যকার পারস্পরিক সম্পর্ক (Relationship) বিবেচনা করে, যা ফিল্টার মেথড পারে না।



Wrapper Method-এর ৩টি প্রধান টেকনিক:

A. Exhaustive Feature Selection (সেরা কিন্তু ব্যয়বহুল 🧠)

এটি সব সম্ভবপর কম্বিনেশন চেক করে দেখে।

- কিতাবে কাজ করে: ধরো তোমার কাছে ৩টি ফিচার (A, B, C) আছে। এই মেথডটি {A}, {B}, {C}, {A,B}, {A,C}, {B,C}, এবং {A,B,C}—এই সবগুলো কম্বিনেশন নিয়ে মডেল চালিয়ে দেখবে কোনটা সেরা।
- সমস্যা: যদি n টি ফিচার থাকে, তবে 2^n সংখ্যক কম্বিনেশন তৈরি হয়। ফিচার সংখ্যা বাড়লে এটি চালানো অসম্ভব হয়ে পড়ে (Computational Expense)।

B. Sequential Forward Selection (শূন্য থেকে শুরু 🚀)

এটি একটি "Empty Set" বা খালি ঝুড়ি নিয়ে শুরু হয়।

- **Step 1:** প্রথমে প্রতিটি ফিচার একা একা চালিয়ে দেখে কোনটা সবচেয়ে বেশি accuracy দেয়। ধরো 'Age' সেরা হলো।

- **Step 2:** এবার 'Age'-এর সাথে বাকি ফিচারগুলো একটা একটা করে যোগ করে দেখে (যেমন: Age+Income, Age+Gender) কোনটা রেজাল্ট আরও বাড়ানো।
- কখন থামবে: যতক্ষণ পর্যন্ত accuracy বাড়বে, ততক্ষণ এটি ফিচার যোগ করতে থাকবে।
- উদাহরণ: একটি ক্রিকেট টিম বানানো—প্রথমে সেরা প্লেয়ার নিলে, তারপর তার সাথে মানানসই দ্বিতীয় সেরা প্লেয়ার যোগ করলে।

C. Sequential Backward Elimination (সব নিয়ে শুরু)

এটি ফরওয়ার্ড সিলেকশনের ঠিক উল্টো। এটি "Full Pantry" বা সব ফিচার নিয়ে শুরু করে।

- **Step 1:** সব ফিচার (A, B, C, D) নিয়ে মডেল চালায়।
- **Step 2:** এরপর একটি একটি করে ফিচার বাদ দিয়ে দেখে কোন ফিচারটি বাদ দিলে accuracy-তে কোনো ক্ষতি হচ্ছে না বা রেজাল্ট ভালো হচ্ছে। সবচেয়ে ফালতু বা ক্ষতিকর ফিচারটি আগে ছুঁড়ে ফেলে দেওয়া হয়।
- কখন থামবে: যখন দেখা যাবে কোনো ফিচার বাদ দিলেই accuracy কমে যাচ্ছে, তখন বাদ দেওয়া থামিয়ে দেয়।

Wrapper Methods-এর সুবিধা ও অসুবিধা

সুবিধা (Pros)	অসুবিধা (Cons)
High Accuracy: মডেলের ওপর ভিত্তি করে সেরা ফিচার খুঁজে পায়।	Very Slow: বার বার মডেল চালাতে হয় বলে অনেক সময় লাগে।
Feature Interaction: ফিচারের মধ্যকার সম্পর্ক বুঝতে পারে।	Overfitting Risk: ডেটাসেট ছোট হলে মডেল শুধু ওই ডেটার জন্যই সেরা হতে চায়।

মনে রাখার শর্টকাট (Exam Hack)

- **Filter:** স্ট্যাটিস্টিক্যাল টেস্ট (Fast, No ML model).
- **Wrapper:** মডেল চালিয়ে ফিচারের "ডেট" দেখা (Slow, High accuracy).
- **Forward:** ০ থেকে শুরু করে যোগ করা।
- **Backward:** সব থেকে শুরু করে বিয়োগ করা।

Topic: Embedded Methods

১. Embedded Methods কী?

সহজ কথায়, এখানে ফিচার সিলেকশন আলাদা কোনো ধাপ নয়; বরং এটি মডেল ট্রেনিং হওয়ার সময় নিজে নিজেই ঘটে। অর্থাৎ, অ্যালগরিদম যখন শিখতে থাকে, তখনই সে সিদ্ধান্ত নেয় কোন ফিচারটি রাখা দরকার আর কোনটি বাদ দেওয়া দরকার।

- এটি **Filter** মেথডের মতো দ্রুত এবং **Wrapper** মেথডের মতো নির্ভুল।

প্রধান ২ ধরনের Embedded Methods:

A. Regularization (রেগুলারাইজেশন)

এটি একটি গাণিতিক পদ্ধতি যেখানে মডেলের **Loss Function** - মেশিন লার্নিং মডেল যখন কোনো কিছু প্রেডিক্ট করে, তখন সেই প্রেডিকশনটা আসল উত্তরের তুলনায় কতটা ভুল হয়েছে, তা যে ফাংশনের মাধ্যমে বের করা হয় তাকেই **Loss Function** বলে। এর সাথে একটি পেনাল্টি (Penalty) যোগ করা হয়।

- মূল কাজ: যেসব ফিচারের ওজন (Coefficient) অনেক বেশি বা অপ্রয়োজনীয়, সেগুলোকে এটি পেনাল্টি দেয় বা মান কমিয়ে দেয়।
- ৩টি প্রধান টেকনিক:
 1. **Lasso Regularization (L1):** এটি জাদুকরী! এটি অপ্রয়োজনীয় ফিচারের Coefficient একদম ০ (zero) করে দেয়। ফলে ওই ফিচারগুলো মডেল থেকে পুরোপুরি বাদ পড়ে যায়। (Feature selection সরাসরি করে)।

A. Regularization (Lasso & Ridge) - উদাহরণসহ ব্যাখ্যা

মনে করো, তুমি একটি বাড়ির দাম (House Price) প্রেডিক্ট করার জন্য একটি মডেল বানাচ্ছে। তোমার কাছে অনেকগুলো ফিচার আছে:

1. বাড়ির আয়তন (Square feet)
2. বেডরুমের সংখ্যা
3. টয়লেটের সংখ্যা
4. বাড়ির দেয়ালের রঙের ব্র্যান্ড (Paint Brand)
5. বাড়ির দারোয়ানের নাম (Security Guard Name)

এখানে 'দেয়ালের রঙের ব্র্যান্ড' বা 'দারোয়ানের নাম' বাড়ির দামের ওপর তেমন প্রভাব ফেলে না।

১. Lasso Regularization (L1):

Lasso দেখবে যে 'দারোয়ানের নাম' বা 'রঙের ব্র্যান্ড' ফিচারের কোনো গুরুত্ব নেই। সে এই ফিচারগুলোর গুরুত্বের মান (Coefficient) একদম ০ (Zero) করে দেবে।

- ফলাফল: মডেল থেকে ওই ফালতু ফিচারগুলো অটোমেটিক ভ্যানিশ বা ডিলিট হয়ে যাবে। শুধু গুরুত্বপূর্ণ ৩টি ফিচার থাকবে। এই কারণেই একে **Feature Selection** বলা হয়।

2. **Ridge Regularization (L2):** এটি ফিচারের মান কমিয়ে দেয় কিন্তু একদম ০ করে না। এটি সব ফিচারকে মডেলে রাখে তবে প্রভাব কমিয়ে দেয়।

২. Ridge Regularization (L2):

Ridge দেখবে যে 'টয়লেটের সংখ্যা' বা 'বেডরুমের সংখ্যা' হয়তো অতটা গুরুত্বপূর্ণ নয় যতটা 'বাড়ির আয়তন'। সে এদের গুরুত্বের মান অনেক ** কমিয়ে দেবে (যেমন ০.০০০১)** কিন্তু কখনোই ০ করবে না।

- ফলাফল: সব ফিচারই মডেলে থাকবে, কিন্তু যেগুলো কম গুরুত্বপূর্ণ তাদের পাওয়ার কমিয়ে দেওয়া হবে যাতে তারা রেজাল্ট নষ্ট না করতে পারে।

3. **Elastic-Net Regularization** হলো এক ধরনের হাইব্রিড পদ্ধতি, যা **Lasso (L1)** এবং **Ridge (L2)**—এই দুইয়ের শক্তিকে একসাথে ব্যবহার করে।

সহজ কথায়, এটি **Lasso**-র মতো অপ্ৰয়োজনীয় ফিচারকে ০ করে দেয় আবার **Ridge**-এর মতো গ্রুপ অফ ফিচারসের মধ্যে ব্যালেন্স বজায় রাখে।

কেন Elastic-Net প্রয়োজন? (The "Why")

১. **Lasso**-র সমস্যা: যদি তোমার ডেটাসেটে অনেকগুলো ফিচার থাকে যারা একে অপরের সাথে গভীরভাবে যুক্ত (**Highly Correlated**), তবে **Lasso** লটারি করার মতো যেকোনো একটা ফিচারকে রাখে আর বাকিগুলোকে ০ করে দেয়। এতে অনেক সময় দরকারি তথ্য হারিয়ে যায়। ২. **Ridge**-এর সমস্যা: এটি কোনো ফিচারকে বাদ দিতে পারে না, শুধু মান কমায়। ফলে অনেক ফিচার থাকলে মডেল জটিল হয়ে যায়।

সমাধান: **Elastic-Net** এই দুই সমস্যার মাঝামাঝি একটা সমাধান দেয়।

Elastic-Net কিভাবে কাজ করে? (The Mechanism)

এর লস ফাংশন (**Loss Function**)-এ দুটি পেনাল্টি টার্ম যোগ করা হয়:

- L1 Penalty:** যা ফিচার সিলেকশন করতে সাহায্য করে (ফিচার ০ করে দেয়)।
- L2 Penalty:** যা কো-ফিশিয়েন্টগুলোর মান কমিয়ে ওভারফিটিং নিয়ন্ত্রণ করে এবং কো-রিলেটেড ফিচারগুলোকে একসাথে হ্যান্ডেল করে।

বাস্তব জীবনের উদাহরণ (Real-life Example):

ধরো তুমি 'মানসিক চাপ' (**Mental Stress**) প্রেডিক্ট করছো। তোমার কাছে ফিচার আছে:

- কাজের সময় (**Working Hours**)
- ঘুমের সময় (**Sleeping Hours**)
- কফির পরিমাণ (**Coffee intake**)
- ল্যাপটপ ব্যবহারের সময় (**Laptop usage**)

এখানে 'কাজের সময়' এবং 'ল্যাপটপ ব্যবহারের সময়' একে অপরের সাথে খুব বেশি যুক্ত (**High Correlation**)।

- Lasso:** হয়তো শুধু 'কাজের সময়' রাখবে আর 'ল্যাপটপ ব্যবহার' ০ করে দিবে।
- Ridge:** দুটোকেই রাখবে কিন্তু ছোট মান দিবে।
- Elastic-Net:** এটি দেখবে এই দুটো ফিচারই গুরুত্বপূর্ণ এবং একে অপরের সাথে যুক্ত। এটি এই দুই ফিচারের একটা সুন্দর সমন্বয় বা ব্যালেন্স তৈরি করবে যাতে প্রেডিকশন নির্ভুল হয়।

কখন Elastic-Net ব্যবহার করবে?

- যখন তোমার ডেটাসেটে ফিচারের সংখ্যা অনেক বেশি (**Big Data**)।
- যখন অনেকগুলো ফিচার একে অপরের সাথে **Correlated** থাকে।
- যখন **Lasso** বা **Ridge** এককভাবে ভালো রেজাল্ট দিচ্ছে না।

পরীক্ষার জন্য নোট: পরীক্ষায় যদি আসে "**Elastic-Net**-এর সুবিধা কী?" উত্তর দিবে— এটি **Lasso**-র **Feature Selection** এবং **Ridge**-এর **Stability** বা স্থায়িত্বকে একসাথে কাজে লাগায়।

B. Tree-Based Algorithms

ডিসিশন ট্রি বা র‍্যান্ডম ফরেস্টের মতো অ্যালগরিদমগুলো ডেটাকে ভাগ (Split) করার সময় সবচেয়ে গুরুত্বপূর্ণ ফিচারগুলোকে গাছের ওপরের দিকে রাখে।

- কিভাবে কাজ করে: মডেল ট্রেনিংয়ের সময় প্রতিটি ফিচারের **Importance Score** বের করা হয়। যেসব ফিচারের স্কোর বেশি, মডেল সেগুলোকে রাখে, বাকিগুলো ইগনোর করে।

মনে করো, তুমি প্রেডিক্ট করতে চাও "একজন কাস্টমার কি আইফোন (iPhone) কিনবে কি না?" তোমার কাছে ডেটা আছে:

1. কাস্টমারের মাসিক ইনকাম (Monthly Income)
2. কাস্টমারের বয়স (Age)
3. কাস্টমারের প্রিয় রঙ (Favorite Color)

মডেল যখন **Decision Tree** তৈরি করবে, সে দেখবে সবচেয়ে গুরুত্বপূর্ণ হচ্ছে 'মাসিক ইনকাম'।

- কিভাবে কাজ করে: * গাছটির একদম শুরুতে (Root Node) সে **Income** দিয়ে ভাগ করবে। (যেমন: ইনকাম ১ লাখের বেশি কি না?)
 - এর পরের ধাপে সে হয়তো **Age** দিয়ে ভাগ করবে।
 - সে দেখবে যে 'প্রিয় রঙ' দিয়ে ভাগ করলে প্রেডিকশনে কোনো লাভ হচ্ছে না। তাই সে ওই ফিচারটিকে গাছের নিচের দিকে রাখবে অথবা একদম ব্যবহারই করবে না।
- **Feature Importance Score:** মডেল ট্রেনিং শেষে তোমাকে একটি স্কোর দিবে। যেমন:
- Income: 0.85 (খুবই গুরুত্বপূর্ণ)
- Age: 0.12 (মোটামুটি গুরুত্বপূর্ণ)
- Color: 0.03 (একেবারেই দরকার নেই)

এই স্কোরের ওপর ভিত্তি করে তুমি সহজেই অপ্রয়োজনীয় ফিচার (Color) বাদ দিয়ে দিতে পারো।

সংক্ষেপে মনে রাখার উপায়:

- **Lasso (L1):** অপ্রয়োজনীয় ফিচারের গলা কেটে ফেলে (০ করে দেয়)।
- **Ridge (L2):** ফিচারের পাওয়ার কমিয়ে দেয় কিন্তু বাঁচিয়ে রাখে।
- **Tree-based:** সবচেয়ে প্রয়োজনীয় ফিচারকে গাছের মগডালে (সবার উপরে) জায়গা দেয়।

Feature Selection Cheat Sheet (The Final Table)

তোমার স্লাইডের একদম শেষে একটি টেবিল দেওয়া আছে যা পরীক্ষার জন্য "সবচেয়ে বেশি ইম্পোর্ট্যান্ট"। কোন ধরনের ডেটার জন্য কোন টেস্ট করবে তা নিচে সাজিয়ে দেওয়া হলো:

Input	Output	Feature Selection Model
Numerical	Numerical	• Pearson's correlation coefficient
		• Spearman's rank coefficient
		• Trace ratio criterion
		• Mutual information
Numerical	Categorical	• ANOVA correlation coefficient
		• Kendall's rank coefficient
Categorical	Numerical	• Kendall's rank coefficient
		• ANOVA correlation coefficient
Categorical	Categorical	• Chi-Squared test (contingency tables).
		• Mutual Information.